

01 Equality, hashing & references

CS201 MIDTERM 1 | Decide identity, contents, and hash separately

Three different questions

Expression	Meaning
<code>a == b</code>	Same object/reference (primitives: same value)
<code>a.equals(b)</code>	Equality rule defined by the class
<code>a.hashCode()</code>	Hash value; not a unique object ID

`Object.equals` uses identity unless overridden. `String` compares characters; `List` compares elements in order. Arrays inherit identity equality: use `Arrays.equals(a,b)` for array contents. `==` cannot be overridden.

Aliasing versus a new object

```
String s = new String("Duke");
String alias = s;
String t = new String("Duke");
// s == alias      true
// s == t          false
// s.equals(t)     true
// same hash for s,t: true
```

Assignment copies a reference. It does not copy the object. Two calls to `new` create two distinct objects, even if fields match.

Arrays and List views

```
String[] a = {"cat", "dog"};
String[] b = {"cat", "dog"};
String[] alias = a;
// a == b                false
// a.equals(b)           false
// Arrays.equals(a,b)    true
// a == alias            true
List<String> view = Arrays.asList(a);
view.set(0, "owl"); // a[0] is "owl"
// view.add("fox"); // unsupported
```

`Arrays.asList(array)`: fixed-size, array-backed view. `new ArrayList<>(list)`: separate, resizable list; element references are shared (shallow copy). Replacing a slot in the copy does not replace a slot in the original.

Hash implications: memorize this table

Assume non-null objects and a correct `equals/hashCode` implementation. ND = cannot determine.

Given	Question	Result
same hash	<code>equals?</code>	ND
different hash	<code>equals?</code>	False
<code>equals true</code>	<code>== ?</code>	ND
<code>equals true</code>	same hash?	True
<code>equals false</code>	same hash?	ND
<code>== true</code>	<code>equals?</code>	True
<code>!= true</code>	<code>equals?</code>	ND

Equal contents imply equal hashes. Different hashes imply unequal contents. Same hashes can be a collision; unequal contents can have the same hash.

```
String a = "Aa", b = "BB";
// a.equals(b)           false
// a.hashCode()          2112
// b.hashCode()          2112
```

P0: Person201 equality

Local `Person201.equals` compares name, latitude, longitude, eatery. Distinct objects with identical fields: `p == q` is false, `p.equals(q)` is true; `p != q` is true, `!p.equals(q)` is false.

Local source caveat: `Person201` overrides `equals` but not `hashCode`. Content-equal instances are not guaranteed to have equal hashes. Inspect supplied class code before relying on the contract. Its cast-based `equals` also assumes a non-null `Person201` argument.

Printing is a method call

```
System.out.println(p); // toString()
System.out.printf("%s", p); // same
System.out.println("P: " + p); // same
System.out.println(p.name());
// name() returns the name String
```

Overridden `toString()` determines the text. It returns a `String`; printing happens in `println/printf`. Default `Object` text is `class-name@hex-hash`, not a guaranteed memory address. Printing a null object reference gives "null"; calling a method on null throws.

02 Big O & operation costs

Count iterations, then include the work done inside each iteration

A reliable three-step method

1. Define the input sizes. 2. Count each operation/call and its cost. 3. Add sequential costs; sum body costs over iterations. Keep the dominant term. Use the tight growth class intended by exam choices.

Loop pattern	Time (O(1) body)
i++ or i += constant to N	O(N)
i *= 2 or i /= 2	O(log N)
i*i <= N, i++	O(sqrt(N))
Two separate N-loops	O(N), not O(N^2)
Two nested N-loops	O(N^2)
j = i+1 to N inside i-loop	O(N^2)
Log N outer; N inner	O(N log N)
Outer i doubles; inner i times	O(N): 1+2+4+...
Outer to sqrt(N); inner i times	O(N): 1+...+sqrt(N)

Runtime is not the returned value

```
int sum = 0;
for (int k=1; k<=n; k++) {
    sum += k+k;
}
return sum;
// Time: O(n)
// Value: n*(n+1), ignoring overflow
```

A loop can return a huge number with little work, or scan many inputs and return zero. Analyze executed operations, not the answer magnitude.

Constant factors do not change Big O

N^2 versus $N(N-1)/2$: both quadratic. Three sequential scans: $3N$ is linear. Loops to $N/100$ remain $O(N)$. Deleting one append in a $\sqrt{N}$ divisor loop leaves $O(\sqrt{N})$; correctness/output may change.

Common sums: value versus work

Sum	Formula / growth
$1+2+\dots+N$	$N(N+1)/2 = \Theta(N^2)$
$0+1+\dots+(N-1)$	$N(N-1)/2 = \Theta(N^2)$
$1^2+2^2+\dots+N^2$	$N(N+1)(2N+1)/6 = \Theta(N^3)$
$1^3+\dots+N^3$	$[N(N+1)/2]^2 = \Theta(N^4)$
$1+2+4+\dots+2^m$	$2^{m+1}-1$; if $2^m=N$: $2N-1$
$1+1/2+\dots+1/N$	$\Theta(\log N)$

Adding i (or i^2) once on each of N iterations still takes $O(N)$ time in the course arithmetic model. If the body RUNS i times, total work is $\text{sum } i = O(N^2)$. Formula growth alone does not determine runtime.

Core data-structure operations

Operation	Cost
ArrayList get/set/size	$O(1)$
ArrayList append	$O(1)$ amortized
One append requiring resize	$O(N)$
Middle insertion/removal	$O(N)$ shifting
ArrayList copy/addAll	$O(N)$
contains/indexOf on list	$O(N)$ scan
HashMap get/put; HashSet add	$O(1)$ expected*
TreeSet add/contains	$O(\log N)$
Sort N bounded-cost items	$O(N \log N)$

*Course assumption: good hashing and bounded-cost keys. A k -element List key needs $O(k)$ hashing. List equality can examine k elements. N appends take $O(N)$ total despite occasional $O(N)$ resizes.

Strings grow: count copied characters

```
String ret = "";
for (int i=0; i<n; i++) {
    ret += "word ";
}
// Copy 1+2+...+n words: O(n^2)
```

Bounded-length word appends via `String +=` cost $O(n^2)$. If each appended base has length $O(n)$, total can be $O(n^3)$. Use `StringBuilder`, or collect words and `String.join` once: $O(\text{total output characters})$.

Insertion versus replacement

duplicateB: copy N words, addAll N words, then N iterations of get/set: $O(N)$. Work backward to avoid overwriting unread input. duplicateA: N iterator insertions in an ArrayList shift later elements: $O(N^2)$. Same loop count, different body cost.

Timing and boundary checks

Doubling N : linear time roughly $\times 2$; quadratic $\times 4$; cubic $\times 8$. Timings can be noisy: verify against code. For positive N , prime divisor output can be $[1, N]$ even though $\sqrt{N}$ candidates are checked.

int overflow: `Integer.MAX_VALUE + 1` wraps to `Integer.MIN_VALUE`. Do not infer mathematical growth of the return value from runtime. Unless asked about overflow, loop analyses use the course arithmetic model.

03 Constructors, override & tracing

Constructor = initialization | Override = inherited method behavior

One example: trace this exact call

```
class Person {
    private String name;
    private int age;
    public Person(String name, int age) {
        System.out.println("A");
        this.name = name;
        this.age = age;
    }
    public Person(String name) {
        this(name, 0);
        System.out.println("B");
    }
    @Override
    public String toString() {
        return name + " (" + age + ")";
    }
}
Person p = new Person("Amy");
System.out.println(p);
// Output: A, then B, then Amy (0)
```

One object. The one-argument constructor delegates to the two-argument constructor. It prints A and initializes fields, then returns; the original constructor prints B. println uses toString. Age is 0 because this(name,0) supplies 0.

Constructor rules

Constructor name = class name; no return type (not even void). Different parameter lists overload constructors. this.field means the current object's field; this(...) calls another constructor in this class; super(...) calls a parent constructor. For CS201, put constructor delegation first.

Constructors are not inherited or overridden. Java supplies an automatic no-argument constructor only if no constructors are declared. A subclass constructor implicitly calls super() if it has no explicit delegation; a matching accessible parent constructor must exist.

```
public CelestialBody(double mass,
                      String filename) {
    this(0,0,0,0,mass,filename);
} // delegates to six-arg constructor
```

Override versus overload

Override: implement an inherited instance method with matching name and parameter types; compatible return type and access are required. Overload: same name, different parameter lists. @Override is an optional compiler check; it does not create an override.

```
public boolean equals(Object other)
// Overrides Object.equals
public boolean equals(Person other)
// Overloads; different parameter type
// @Override on this overload: error
```

Spelling and capitalization matter: toString, not tostring. Correct overriding works without the annotation. Static methods are class methods, not dynamically overridden instance methods.

Dynamic dispatch: P1

```
BaseMarkovModel m =
    new HashMarkovModel(2);
m.generate(10);
// inherited generate calls getFollows
// HashMarkovModel.getFollows runs
```

The actual object type chooses an overridden instance implementation. The variable's declared type determines which members are accessible at compile time.

Private fields and Scanner

Code inside CelestialBody can access b.myXPos even when b is another instance. Unrelated external code uses getters. java.lang is imported automatically: Math and String need no explicit import.

Arguments evaluate left to right. Each Scanner nextDouble()/next() consumes one token. nextDouble accepts "25" as 25.0. An unused radius read still consumes its token; removing the read shifts all later data.

Trace explicit AND implicit calls

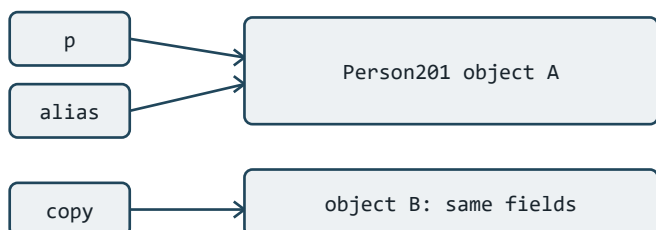
```
map.putIfAbsent(elt, elt.hashCode());
// 1. explicit hashCode for VALUE
// 2. map hashes elt as KEY
// 2 calls -> 2 diagnostic lines
```

If hashCode prints with "%s", that formatting calls toString. One hashCode call prints one line in Inner. putIfAbsent returns the old value or null, not a boolean. containsKey + explicit hashCode + put adds lookup work (up to 3 calls; empty-map shortcuts vary).

04 P0: objects, distances & loops

Follow the picture, use the code pattern, check the boundary

One object or two? Follow the arrows



Compare	Result
p == alias	true: object A
p == copy	false: A versus B
p.equals(copy)	true if all four fields match

Person201: field -> public getter

Private field	Getter
name / eatery	name() / eatery()
latitude / longitude	latitude() / longitude()

```
new Person201(name,lat,lon,eatery);
new Person201(); // default values
System.out.println(p); // toString()
System.out.println(p.name()); // name
```

Class boundary: outside Person201 use getters. Inside it, access to another Person201's private fields is allowed.

Distance: caller and argument

Call	Meaning
distance(p,q)	Static utility; two inputs
p.distanceFrom(q)*	this = p; other = q
q.distanceFrom(p)*	this = q; other = p
distance(p,q) = distance(q,p)	Same value; self-distance = 0

```
// *Hypothetical method in Person201:
public double distanceFrom(Person201 q) {
    return Person201Utilities.distance(this,q);
}
```

*Local P0 uses the static utility; F24 adds distanceFrom; F26 practice omits that premise. Distance uses coordinates and kilometers (EARTH_RADIUS_KM), despite older comments saying miles.

Read data -> scan once

Step / task	Time
File or URL -> objects -> array	O(N)
Count one eatery / nearby people	O(N)
E eateries, each scans N people	O(EN) + set work

All unique pairs: triangular work

N=4: check only cells where j > i

	0	1	2	3
0	-	X	X	X
1	-	-	X	X
2	-	-	-	X
3	-	-	-	-

3 + 2 + 1 = 6 pairs here. General: $N(N-1)/2 \rightarrow O(N^2)$. Starting j at 0 adds self/repeated pairs; still quadratic.

```
double max = -1;
Person201 a = null, b = null;
for (int i=0; i<data.length; i++) {
    for (int j=i+1; j<data.length; j++) {
        double d = Person201Utilities
            .distance(data[i],data[j]);
        if (d > max) {
            max = d;
            a = data[i]; b = data[j];
        }
    }
} // a,b need a valid pair (N >= 2)
```

Min/max: choose a matching start value

Goal	Initialize	Update
Minimum	very large	value < min
Maximum	very small*	value > max
Wrong combination	MAX_VALUE	value > min: no update

*For nonnegative distances, max=-1 permits zero-distance pairs. Null initialization compiles, but dereferencing a still-null best/a/b throws. Unassigned local variables may fail compilation.

Center and threshold: quick decisions

Question	Decision
Center by average distance	N candidates x N scans: $O(N^2)$
Skip self-distance?	Adds 0 either way; sum unchanged
Average over others	Divide by N-1; require N > 1
epsilon > every pair distance	All N counted if self included
Decrease epsilon	Match count cannot increase
Change j start to i or i+1	Big O may stay; correctness may change

Restaurant count: exact String contents

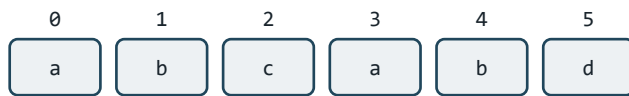
```
int count=0;
for (Person201 p : people)
    if (p.eatery().equals(eatery))
        count++;
// TreeSet: unique, sorted eateries
```

05 P1: windows, maps & generation

N = training tokens | n = generated words | k = context length

See the window and next-word index

k=2; window starts at i=0



context [0,2): [a,b]

next at 2: c

Window (k=2)	Follow word
[a,b] at i=0	c
[b,c] at i=1	a
[c,a] at i=2	b
[a,b] at i=3	d
[b,d] at i=4	None: no next word

Index rule: subList(i,i+k) excludes i+k. Next word = get(i+k). Require i+k < size.

Same request, two lookup paths

Simple: scan each time	Hash: train once, lookup
Find [a,b] windows -> [c,d]	Store [a,b] -> [c,d]
getFollows: O(N)	getFollows: O(1) expected

```
// Simple: collect matching follows
for (int i=0; i+k<seq.size(); i++)
  if (seq.subList(i,i+k)
      .equals(context))
    follows.add(seq.get(i+k));
```

```
// Hash: for each window in training
List<String> key=seq.subList(i,i+k);
myMap.putIfAbsent(key,
    new ArrayList<>());
myMap.get(key).add(seq.get(i+k));
```

Repeated follows stay: [sat,ran,sat] -> P(sat)=2/3. Context length k is not follow-list length. Do not use a Set for follows.

One generation step: follow the arrows

Step	Example
1. Current context	[a,b]
2. getFollows(context)	[c,d]
3. Randomly choose next	c
4. Append output word	... c
5. Drop first, append next	[b,c] -> repeat

Start from getRandomContext. Stop at END or n words; join output once. Actual subclass selects getFollows via dynamic dispatch.

Runtime lookup: fixed k, bounded words

Method	Time
trainText / trainDirectory*	O(N)
getRandomContext / new context	O(1)
differentContexts / getSequence	O(N)**
Simple lookup / randomNextString	O(N)
Hash processTraining	O(N) expected
Hash lookup / randomNextString	O(1) expected
Simple generate(n)	O(nN)
Hash generate(n)	O(n) expected

*Fresh training: read/tokenize -> k START + k END per text -> store tokens -> processTraining. N includes padding. Repeated Hash training rebuilds accumulated tokens. **differentContexts uses expected hashing; getSequence copies.

If k varies: include key and copy cost

Operation	Time
Simple getFollows	O(Nk) worst case
Hash lookup / randomNextString	O(k) expected
Hash train / differentContexts	O(Nk) expected
Simple generate(n)	O(nNk) worst case
Hash generate(n)	O(nk) expected
Random context / new context	O(k) copies

Context update: [a,b] + c -> [b,c]

```
context = new ArrayList<>(
    context.subList(1,context.size()));
context.add(next);
context = List.copyOf(context);
// copy k-1 -> append -> copy k
```

Operation	Remember
ArrayList.subList	O(1) view; shares backing list
List.copyOf	Copies references; immutable result
Mutate a map key?	Avoid: hash/equality can change

Output building can dominate runtime

Build output	Bounded words
List.add + one String.join	O(n)
StringBuilder append	O(n) amortized
ret += next + " " repeatedly	O(n^2): copy 1+...+n

randomNextString includes getFollows; only random-index selection is O(1). Same training/order/seed and follow-list order should produce the same text in Simple and Hash.

06 Collection patterns & APT code

Anonymous: letter supply | CounterAttack: exact word frequencies

Group values by a key: Map<K,List<V>>

```
Map<String,List<String>> group=new HashMap<>();
for (String uni : universities) {
    String state = stateOf(uni);
    group.putIfAbsent(state,new ArrayList<>());
    group.get(state).add(uni);
}
```

Create a list only for a new key; always append. Repeated put(key,new list) erases previous values. keySet = unique keys; a Set of values removes repeated values.

Largest grouped list

```
int largest=-1; String best=null;
for (String key : group.keySet()) {
    int size = group.get(key).size();
    if (size > largest) {
        largest=size; best=key;
    }
}
```

N keys -> O(N) expected; size() is O(1), not a scan. Strict > keeps the first tied max. HashMap order is not guaranteed.

Filter a list, then return an array

```
List<String> ret = new ArrayList<>();
for (String s : words) {
    if (s.length() >= minLength) ret.add(s);
}
return ret.toArray(new String[0]);
```

Inclusive threshold: >=. Longest length 7 -> threshold 8 gives empty output. Array: .length(); String: .length(); List: .size(). Indexes: 0..length-1.

maxLetter: char is a numeric index

```
public static void maxLetter(String s) {
    int[] counts=new int[256];
    for(char ch:s.toLowerCase().toCharArray())
        counts[ch]++; // BLANK 1
    int max=0;
    for(char ch='a'; ch<='z'; ch++)
        if(counts[ch] > max) // BLANK 2
            max=counts[ch];
    for(char ch='a'; ch<='z'; ch++)
        if(counts[ch] == max)
            System.out.printf(
                "char '%s' occurs %d times\n",ch,max);
}
```

char	Numeric value	Array slot
'a'	97	counts[97]
'b'	98	counts[98]
'z'	122	counts[122]

ch++: a -> b -> ... -> z (26 iterations). Counts start at 0. First pass tallies characters; second finds max; third prints matching letters. For "the quick brown fox jumps over the lazy dog": o, 4.

Time: O(|s|+26+26)=O(|s|). Only a-z are considered for max. This 256-slot array assumes character codes <256; it does not cover all Unicode. Ties print multiple letters; no letters prints all 26 at zero.

Anonymous: can the letters make it?

Count headline characters once; count each message separately. Accept a message iff every letter a-z has need <= have. Below is the same algorithm compacted with a frequency helper.

```
// freq(text): count all characters
Map<Character,Integer> freq(String s) {
    Map<Character,Integer> m =
        new HashMap<>();
    for (char c : s.toLowerCase()
        .toCharArray())
        m.put(c,m.getOrDefault(c,0)+1);
    return m;
}
boolean canMake(String s,
    Map<Character,Integer> have) {
    Map<Character,Integer> need = freq(s);
    for (char c='a'; c<='z'; c++)
        if (need.getOrDefault(c,0) >
            have.getOrDefault(c,0))
            return false;
    return true;
}
// howMany(headlines,messages):
Map<Character,Integer> have =
    freq(String.join(" ",headlines));
int count=0;
for (String s : messages)
    if (canMake(s,have)) count++;
return count;
```

Example: headlines ["Ab","a"] supply a:2,b:1. Messages ["AA","aba","bbb"] return 2. Each message reuses the full supply; letters are not consumed across messages. Lowercasing ignores case. Only a-z is checked, so spaces/punctuation do not affect acceptance.

Time: H headline characters (including join separators), M messages, L total message characters: O(H+L+26M) = O(H+L+M) expected. The 26-letter loop is constant per message. Return value is a count from 0 to M.

CounterAttack: count each query word

```
int[] counter = new int[words.length];
String[] tokens = str.split(" ");
for (int i=0; i<words.length; i++) {
    int count=0;
    for (String word : tokens)
        if (word.equals(words[i])) count++;
    counter[i]=count;
}
return counter;
```

Example: str="a b a", words=["a","b","c","a"] returns [2,1,0,2]. Reset count for each query. String.equals is case-sensitive and compares the whole token. Duplicate queries produce repeated counts in the original query order.

Time: Q query words, T tokens, C input characters: O(C+QT) for bounded word lengths; equality may cost up to O(w) for length w, giving O(C+QT_w). Result length is Q, not T. split(" ") uses a single-space delimiter, not general whitespace.